

IO 接口

IO 接口作用

接口可以看成是 CPU 与其他设备的“中介”。其可以实现数据缓冲，以此匹配不同外设工作速度；转换数据串并格式；转换电平。

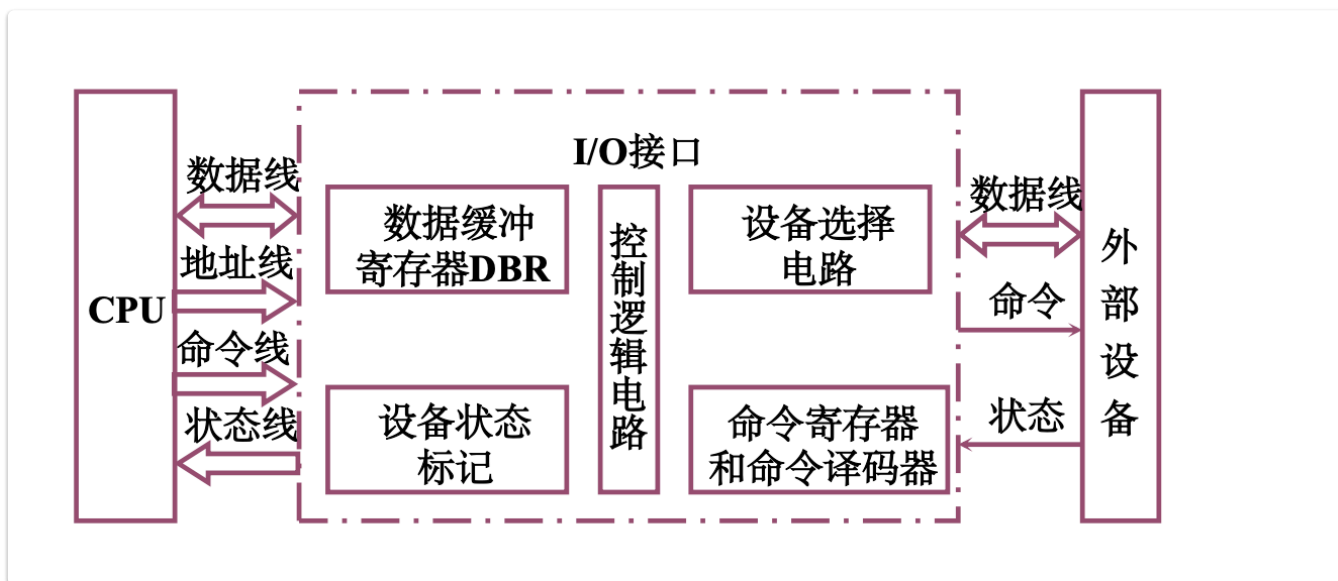
对于 CPU 而言，其作用在于：

- 选择设备
- 将命令传递给设备，以此控制设备
- 得到设备的状态，CPU 通过设备状态决定自己要怎么控制设备
- CPU 通过接口读取设备传来的数据/向设备传数据

对于设备而言，其作用在于：

- 设备通过接口，接受 CPU 的命令
- 通过接口读取 CPU 传来的数据/向 CPU 传数据

IO接口组成



接口类型

- 按数据 **传送方式** 分类：【并行 / 串行】 接口
- 按 **可编程** 分类：【可编程 / 不可编程】 接口
- 按 **通用性** 分类：【通用 / 专用】 接口
- 按数据传送的 **控制方式** 分类：【中断 / DMA】 接口（接下来会讲）

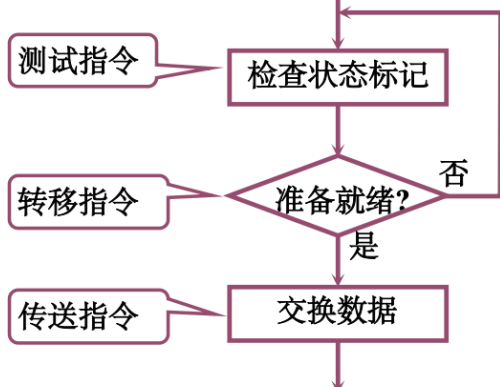
传送控制方式

程序查询方式

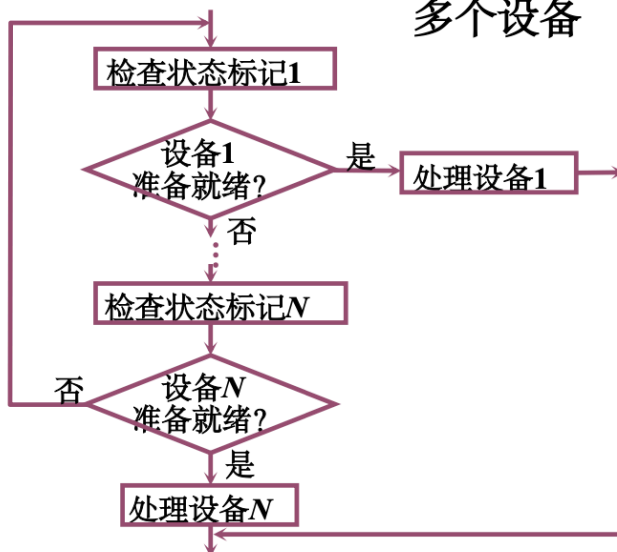
我个人认为这里用“轮询”会更准确

这是一种 **串行** 的工作方式，CPU 会和 IO 串行工作。CPU 会“非常专注”地 **一直询问** 设备的状态，设备没准备好就等待，准备好 CPU 才继续执行。

• 单个设备



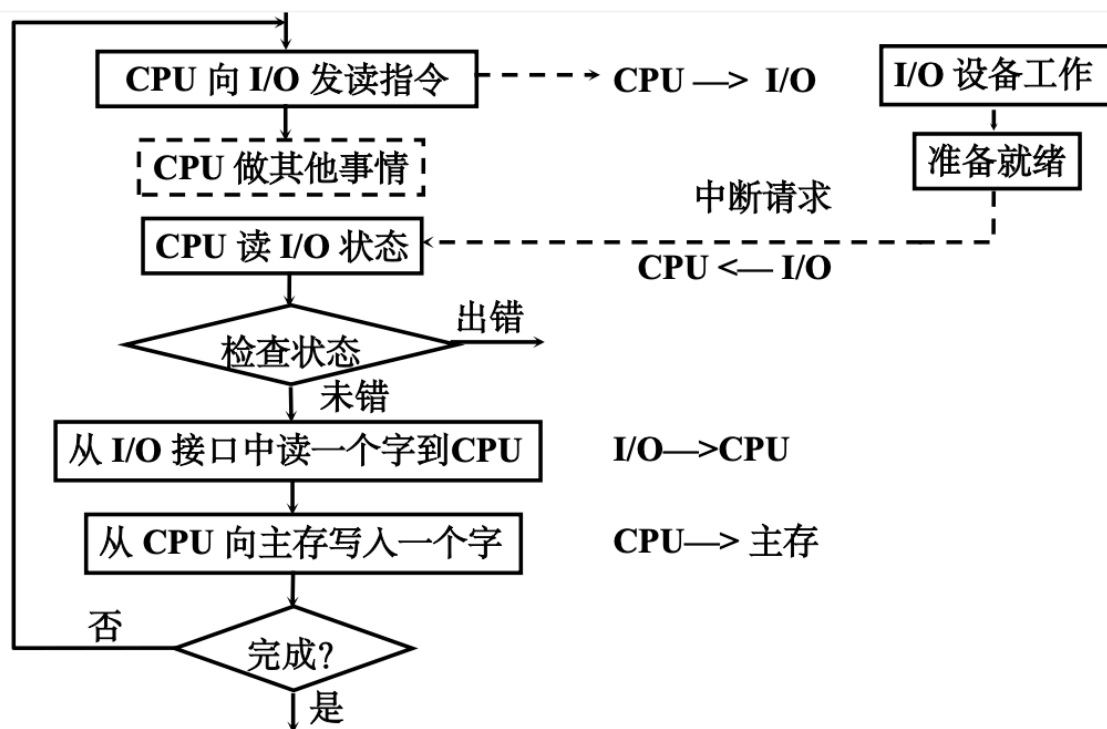
多个设备



多个设备时，CPU 不会只尝试一个设备。但本质没有区别，仍然是串行。

CPU 的时间非常昂贵，这显然不是一个聪明的办法。CPU 理应利用设备准备的时间干点别的。

程序中中断方式



这是一种 **并行** 的工作方式，CPU 和 IO 并行工作。CPU 在向 IO 发出指令之后，去干别的。当 IO 设备准备就绪，它会向 CPU 发出 **中断请求**，来告诉 CPU “我已经准备好了”。CPU 这时候会停下手头的事情，处理 IO，向 IO 发出下一条指令，然后继续忙别的，循环往复。

开销方面：我们节省了 CPU 空等的开销，但会带来 **中断开销 + 中断服务程序** 的运行时间。

这种控制方式，适合 **随机出现** 的服务。如果是高速 IO 设备，可能会带来 **大量** 的开销（👉）。

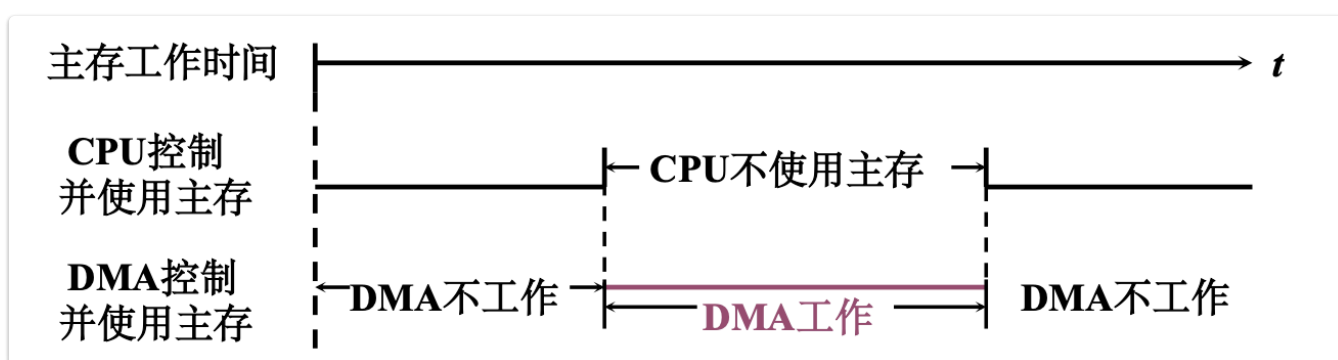
关于 **中断**，后文再进行更详细的解释。我们先看下一一种传送控制方式。

DMA方式

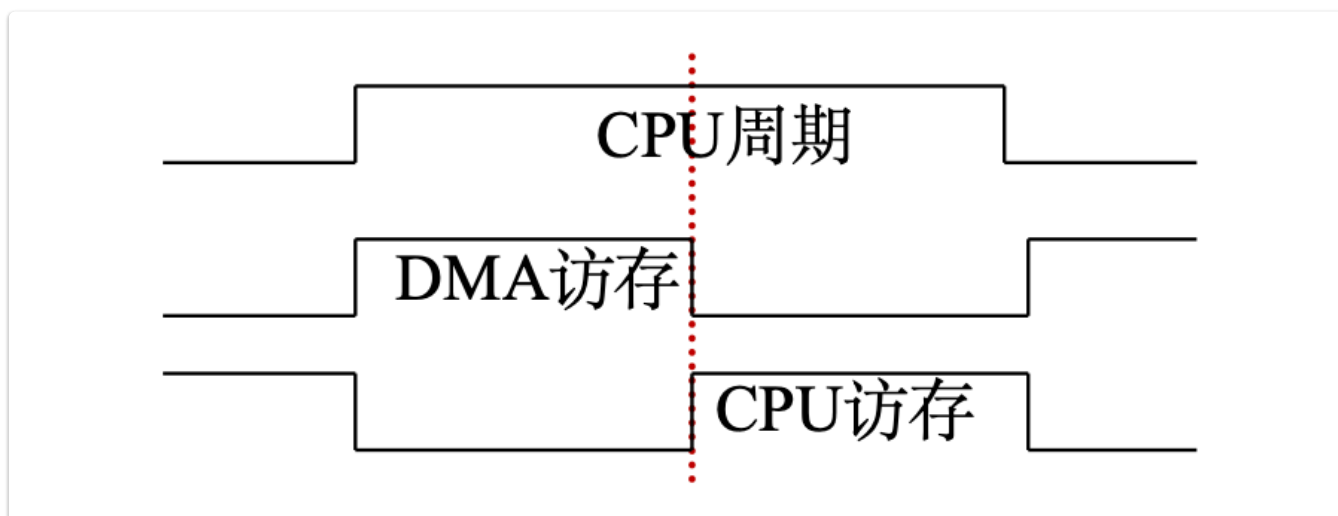
Direct Memory Access。可以将 DMA 看作是 CPU 管理数据传送操作的 **帮手**。CPU 将任务数据传送任务交给 DMA 后，就可以干其他事了。DMA 会负责管理主存和外设的交互。

注意，DMA 和 CPU 都会使用到内存，存在使用权的冲突（**内存争用**）。

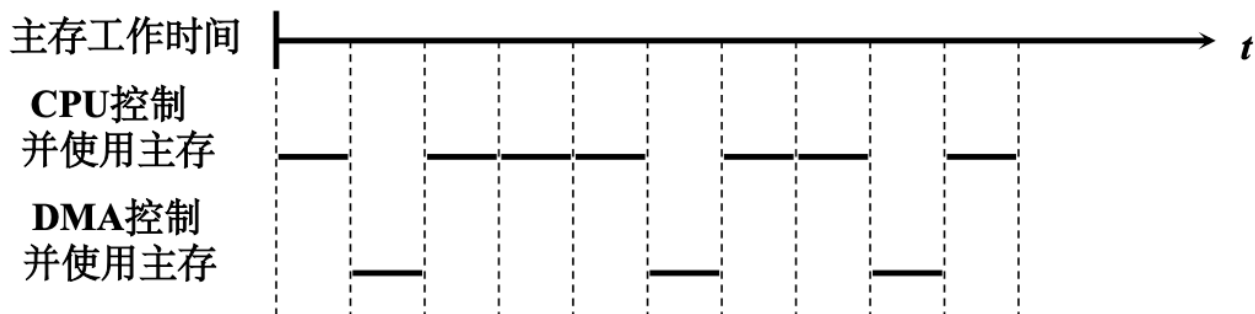
- 解决方法1: **停止 CPU 访问主存**。对于这种方法，如果 DMA 传输周期过长，可能会导致 CPU 长期无法访问内存！同时 DMA 的传输数据时间可能大于访存周期，导致内存访问时间被无意义浪费。



- 解决方法2: **DMA 与 CPU 交替访问**。CPU 前半周期专供 DMA 访存，后半周期专供 CPU 访存。



- 解决方法3: **周期挪用**。DMA要求访存时，CPU暂停一个或多个 **存储周期**，将总线控制权让给 **DMA**，数据传送结束后，CPU继续运行。



1 和 3 有什么区别：1 是 CPU 必须等待 DMA 完全访问完内存后才开始访问内存；3 是让 DMA 一段一段时间地“偷”内存的使用权

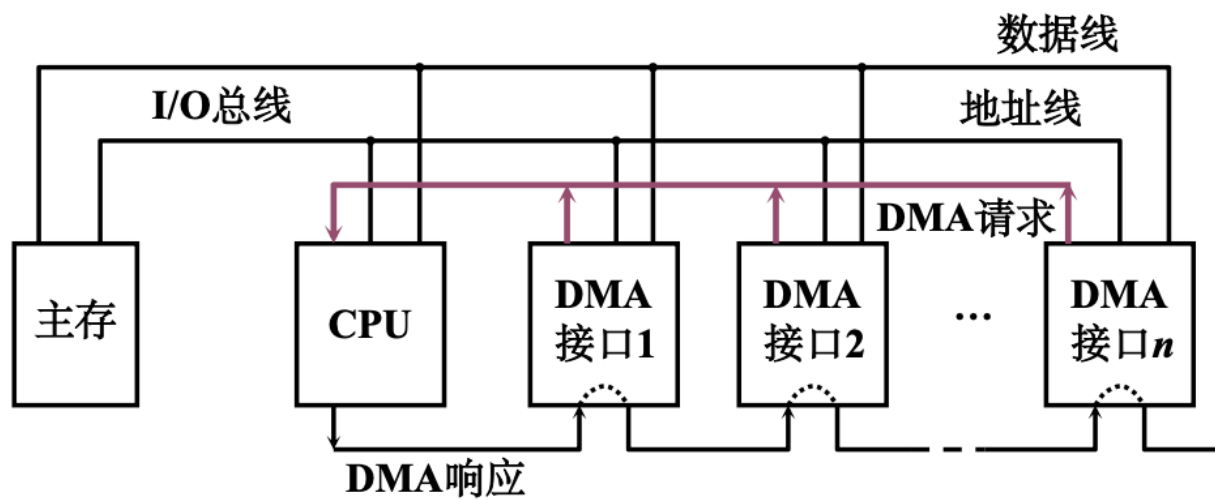
DMA 接口

DMA 接口的功能

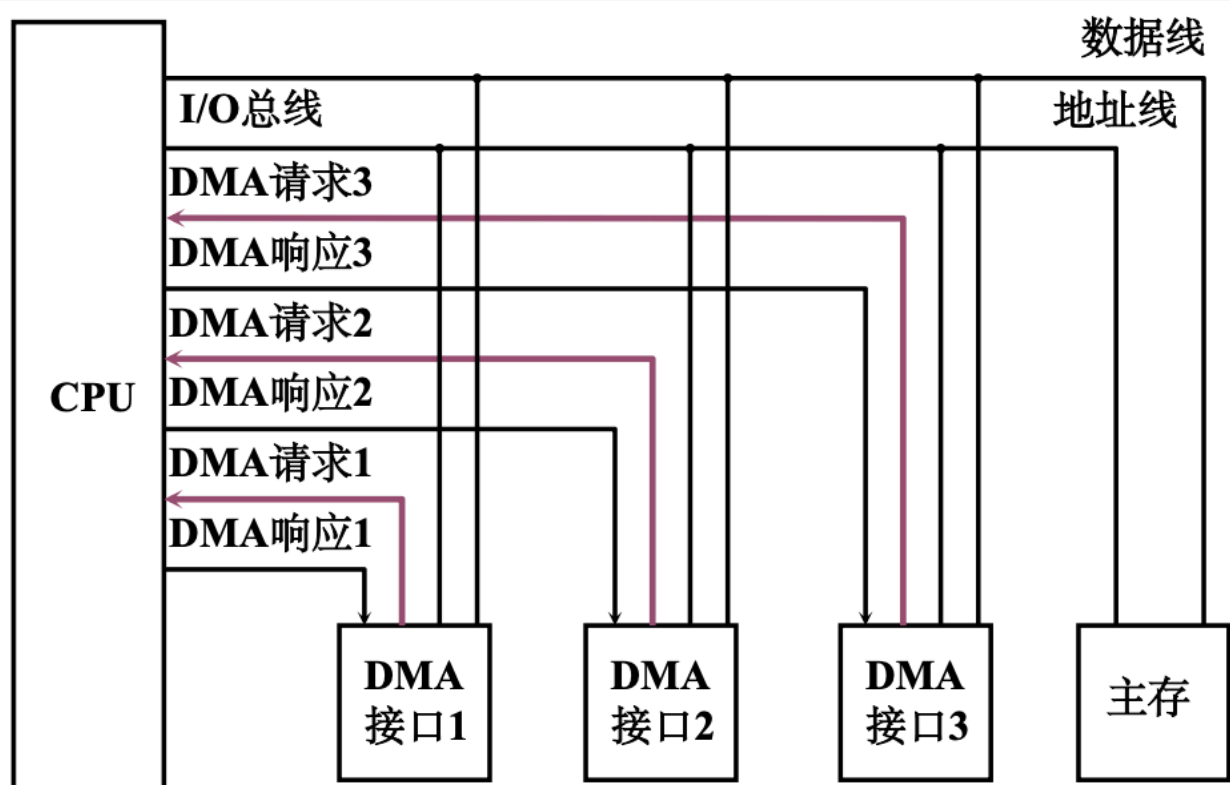
- 向 CPU 申请 DMA 传送
- 处理 总线控制权的转交
- 管理 系统总线、控制 数据传送
- 确定 数据传送的 首地址和长度
 - 更新 传送过程中的数据 地址 和 长度
- DMA 传送结束时，给出操作完成信号

DMA接口与系统可以有以下连接方式：

- 公共连接线的 DMA 请求



- 独立的 DMA 请求



DMA 工作过程

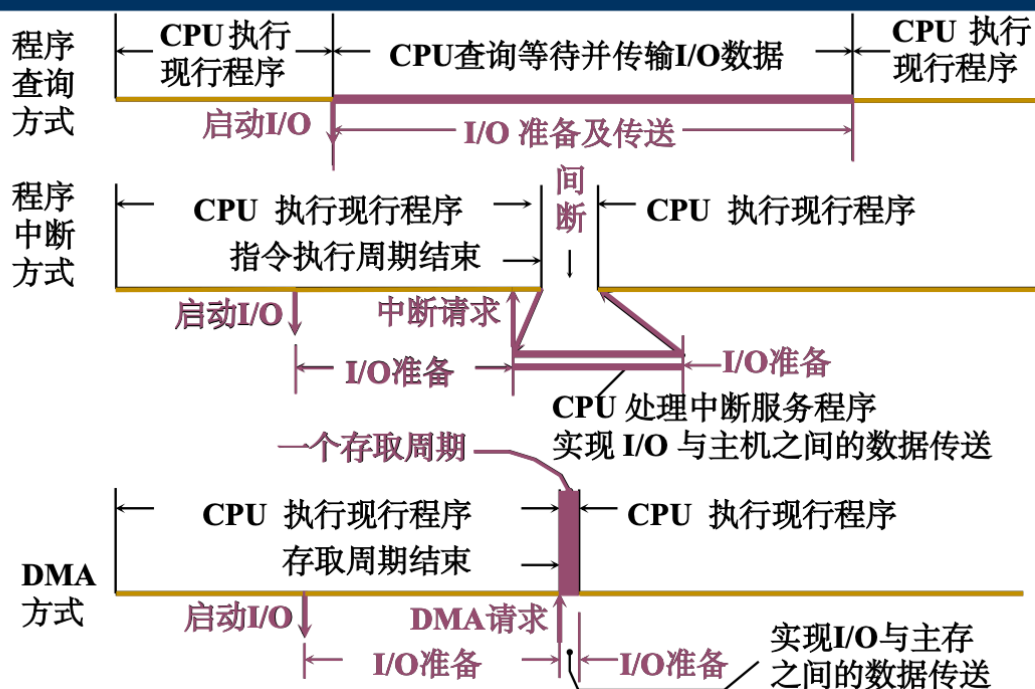
- 主机通过指令发送必要 **传送参数**，启动 DMA
- 数据传送
- **后处理 (结束阶段)**：检验结果是否准确，是否需要用 DMA， **由中断服务程序完成**

比较

DMA方式与程序中中断方式的比较

	中断方式	DMA方式
• 数据传送	程序	硬件
• 响应时机	指令周期后期	存储周期后期
• 处理异常情况	能	不能
• 中断请求	传输数据	后处理
• 优先级	低	高

三种方式的CPU工作效率比较



中断

概念

CPU暂时中止现程序的执行，转去执行为某个随机事件服务的 **中断服务子程序**(Interrupt Service Routine)，处理完后自动**恢复**原程序的执行

产生

- 人为设置：比如说调试的断点
- 程序异常

- 硬件故障
- IO 设备
- 外部事件：比如用键盘中断当前程序

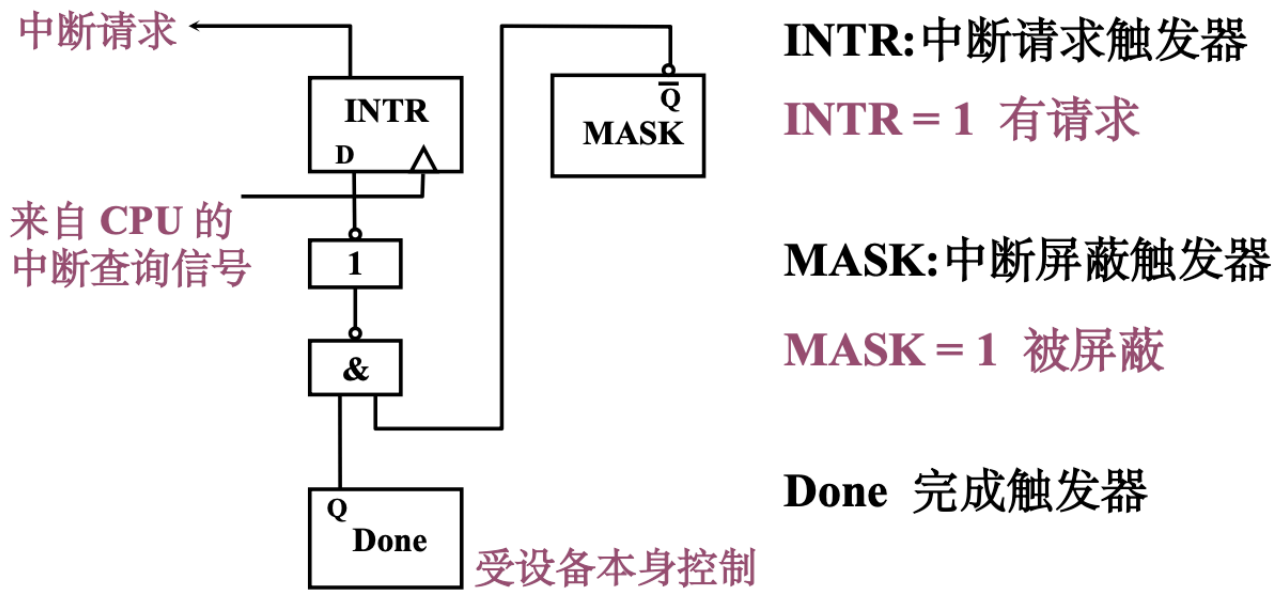
如何实现程序中中断方式

我们现在来试着思考一下如何实现 **程序中中断方式**

触发器

首先我们需要有硬件支持，我们需要可以 **发送 & 接收 & 屏蔽** 中断 的设施。

• 配置中断请求触发器和中断屏蔽触发器



- 一个中断请求源对应一个 **INTR** 中断请求标记触发器
 - **INTR** 分散 在各个中断源的 接口电路中
 - **INTR** 集中在 **CPU** 的中断系统 内
- 多个 **INTR** 触发器组成 **中断请求寄存器**

1	2	3	4	5					<i>n</i>
电	掉	热	过	错	主	溢	阶	非	
					存	读	上	法	
					写	写		除	
					校	验			

中断排队

我们现在可以收到中断请求了，接着需要确定先处理哪一个中断。

- 硬件故障中断属于最高级，其次是程序错误中断
- 非屏蔽中断优于可屏蔽中断
- DMA请求优先于I/O设备传送的中断请求
- 高速设备优于低速设备
- 输入设备的中断优于输出设备
- 实时设备优先于普通设备

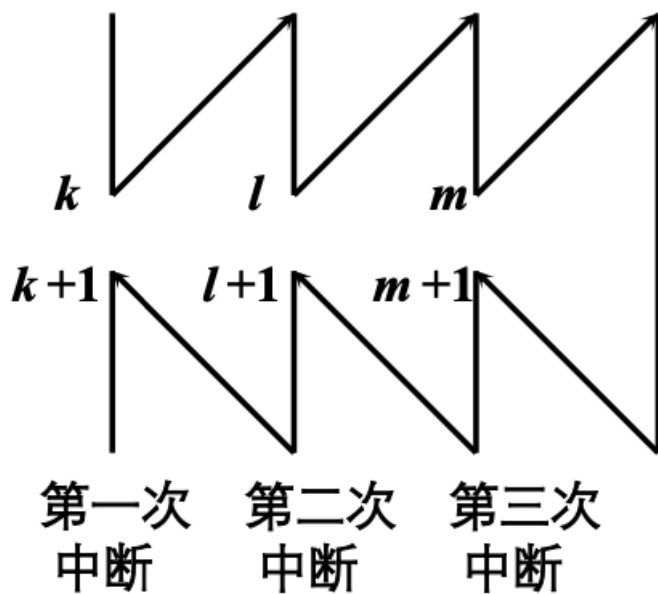
中断排队器 可以用硬件/软件实现。

中断处理

我们还需要确定中断服务程序入口地址，因为不同的中断对应的是不同的入口。我们可以让软件产生这个入口地址；也可以让硬件负责维护中断到入口地址的映射（硬件产生向量地址，再由向量地址找到入口地址）

如果我们在当前中断处理时，选择屏蔽其他中断，此时为 单重中断。单重中断不允许中断现行的中断服务程序。

反之，如果我们并不屏蔽中断（设置了开中断指令），则可能发生 多重中断。这时，我们允许 级别更高 的中断源中断现在的 中断服务程序。



程序断点 $k+1, l+1, m+1$

我们可以通过设置 **屏蔽触发器**，来改变 **处理优先级**。（**响应优先级**不可改变，硬件写死了）

中断服务程序

中断服务程序的处理流程一般如下：

- 保护现场：**中断隐指令**保存程序断点（即 PC），**进栈指令**保护寄存器内容，保存的内容是为了后续恢复现场。
- 中断服务：对中断做出处理
- 恢复现场：**出栈指令**用于恢复现场
- 中断返回：**中断返回指令**